

## Streams (Seqüències)

### Índex

|                                         |    |
|-----------------------------------------|----|
| Streams (Seqüències).....               | 1  |
| 1. Definició.....                       | 1  |
| 2. Character File.....                  | 3  |
| 2.1 FileReader.....                     | 3  |
| 2.2 FileWriter.....                     | 6  |
| 4. Byte File.....                       | 8  |
| 4.1 FileInputStream.....                | 9  |
| 4.2 UTF-8.....                          | 10 |
| 4.2.1 Utilitzant InputStreamReader..... | 10 |
| 4.2.2 Utilitzant FileInputStream.....   | 10 |
| 4.3 FileOutputStream.....               | 12 |
| 5. Throws.....                          | 13 |
| Appendix.....                           | 13 |

### 1. Definició

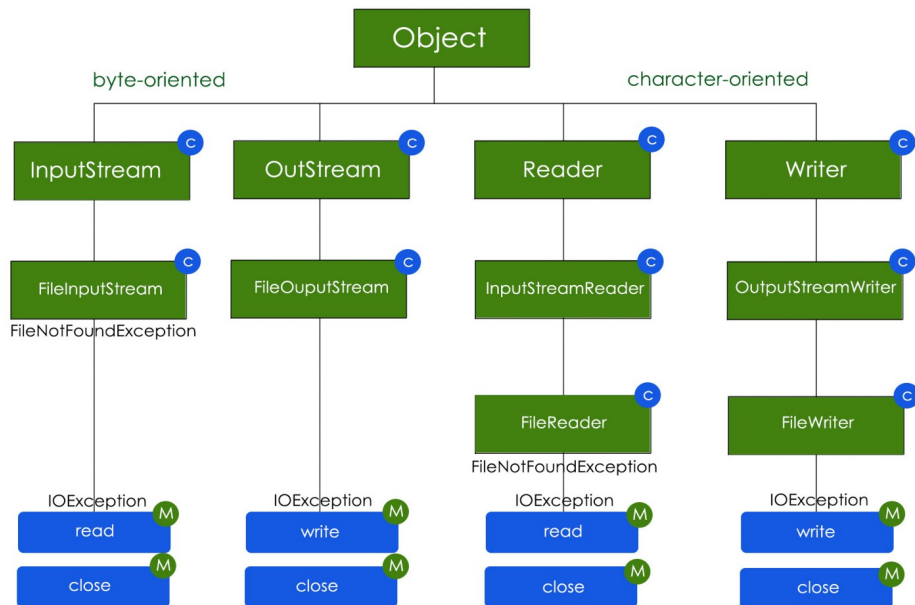
A vegades necessitem emmagatzemar les dades en un fitxer, en comptes de la memòria virtual. D'aquesta manera sempre són accessibles, i no s'esborraran quan s'acabi el programa o s'apagui l'ordinador.

Hi ha dues maners de treballar amb **Streams**:

En canvi el **flux de caràcters** ens serveix només per fitxers de text.

El **Flux de bytes** ens serveix per tot tipus de fitxers, pdf, jpg, documents word, etc.

| Flux de caràcters:                                                                                                                              | Flux de bytes:                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Quan s'envia informació i és necessari llegir-la:</p> <p>ABCD SE SSS<br/>sdd fssaf sfsffs<br/>sd e3ty4 543</p>                               | <p>Quan s'envia informació i no es necessari llegir-la:</p> <p>10010001<br/>01001001<br/>11010101</p>                                                 |
| <p>Classe: <b>FileReader</b></p> <p>Ens permet accedir a la informació d'un fitxer de text</p> <p>Mètodes:<br/><b>read</b><br/><b>close</b></p> | <p>Classe <b>FileInputStream</b>:</p> <p>Ens permet accedir a la informació d'un fitxer de bytes</p> <p>Mètodes:<br/><b>read</b><br/><b>close</b></p> |
| <p>Classe <b>FileWriter</b>:</p> <p>Ens permet escriure informació en un fitxer de text</p> <p>Mètodes:<br/><b>write</b><br/><b>close</b></p>   | <p>Classe <b>FileOutputStream</b>:</p> <p>Ens permet escriure informació en un fitxer de bytes</p> <p>Mètodes:<br/><b>write</b><br/><b>close</b></p>  |



Hem d'importar les classes de *java.io.\** :

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
```

## 2. Character File

### 2.1 FileReader

La classe **FileReader** especificarem la ruta del fitxer:

En cas de tenir una ruta absoluta:

Per Linux: `FileReader fr = new FileReader("/usr/Workspace/M03UF3/dir/file.txt");`  
 Per windows: `FileReader fr = new FileReader("C:\\Users\\Workspace\\M03UF3\\dir\\file.txt");`  
 Una barra invertida "\" per indicar seqüència d'*escape* i una altra per indicar el directori.

En cas de tenir una ruta relativa:

Per Linux: `FileReader fr = new FileReader("dir/file.txt");`  
 Per windows: `FileReader fr = new FileReader("dir\\file.txt");`

En cas d'utilitzar el separador del sistema, vàlid per Linux i Windows:

`FileReader fr = new FileReader("dir" + File.separator + "file.txt");`

Tenim diferents opcions alhora de llegir el contingut d'un fitxer que provenen de diferents classes:

| Mètode                                                     | Exemple                                                                                           | Prové de la classe | Significat                                                                                                                                                                                      |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int read()</code>                                    | <code>int c = fr.read()</code>                                                                    | InputStreamReader  | Llegeix un caràcter en codi ASCII.                                                                                                                                                              |
| <code>int read(char[] chars)</code>                        | <code>char[] chars = new char[10];</code><br><code>int totalChar = fr.read(chars);</code>         | Reader             | Guarda a <code>chars</code> (array de 10 caràcters) els 10 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 10).                        |
| <code>int read(char[] chars, int start, int length)</code> | <code>char[] chars = new char[40];</code><br><code>int totalChar = fr.read(chars, 10, 30);</code> | InputStreamReader  | Guarda a <code>chars</code> (array de caràcters) a partir de la posició 10 els 30 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 30). |

**Exemple 1:** Mostrem per consola el contingut del fitxer caràcter a caràcter

- Instanciem un objecte de la classe **FileReader** que serà la ruta del nostre fitxer de text.
- Degut a que **FileReader** llança una excepció del tipus **FileNotFoundException**, aquesta s'ha de capturar amb un **try-catch**. Aquesta excepció controla si el fitxer no existeix.
- Amb el mètode **read()** llegim el primer caràcter en codi ASCII. Degut a què aquest mètode llança una excepció **IOException**, hem de capturar-la amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Anem llegint fins trobar el codi -1, indicador de final de fitxer.
- Tanquem la lectura amb el mètode **close()**.

```
System.out.println("READ Console output:");
try {
    FileReader file = new FileReader("text.txt");
    int c = file.read();

    while (c!=-1) {
        char cl = (char) c;
        System.out.print(cl);
        c = file.read();
    }
    file.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

**Exemple 2:** Llegim directament tot el fitxer guardant les dades en un array de tipus *char* sabent quina és la mida del fitxer:

```
try {
    File fl = new File("text.txt");           //To be used to know the length of the file in bits
    FileReader fr = new FileReader("text.txt");
    char[] chars = new char[(int)fl.length()]; //an array of chars where data will be saved

    int totalChar = fr.read(chars);           //read the data and save it into array of chars
                                           //totalChar contains the total chars has been read
    String text = new String(chars);          //convert array into String
    System.out.print(text);
    System.out.print("Total chars: " + totalChar);
    fr.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("File can't be read: " + e.getMessage());
}
```

**Exemple 3:** Guardem a la posició *x* de l'array de caràcters de tipus *char* (**chars**), els nombre de caràcters especificat en **length** del fitxer que estem llegint, si executem el següent codi, podrem llegir tot el fitxer però veurem que els 100 primers caràcters són espais en blanc.

```
try {
    int x = 100;
    File fl = new File("text.txt");           //To be used to know the length of the file in bits
    int length = (int)fl.length();
    char[] chars = new char[x + length];

    FileReader fr = new FileReader("text.txt");
    int totalChar = fr.read(chars, x, length); //an array of chars where data will be saved

    String text = new String(chars);          //convert array into String String
    System.out.print(text);
    System.out.print("Total chars: " + totalChar);
    fr.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("File can't be read: " + e.getMessage());
}
```

**Exemple 4:** Guardem els caràcters a un **ArrayList** amb objectes **Character**, posteriorment mostrem per consola el contingut de l'**ArrayList**:

```
List<Character> dataC = new ArrayList<>();
try {
    FileReader file = new FileReader("text.txt");
    int c = file.read();

    while (c != -1) {
        dataC.add(((char)c));
        c = file.read();
    }
    file.close();
    for (Character ch : dataC) {
        System.out.print(ch);
    }
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

**Exemple 5:** Guardem els caràcters a un **ArrayList** amb objectes **Integer**, posteriorment mostrem per consola el contingut de l'**ArrayList**, en aquest cas tenim dues opcions de passar d'**Integer** a **char**:

```
List<Integer> data = new ArrayList<>();
try {
    FileReader file = new FileReader("text.txt");
    int c = file.read();

    while (c != -1) {
        data.add(c);
        c = file.read();
    }
    file.close();
    for (int i = 0; i < data.size(); i++) {
        System.out.print(((char)data.get(i).intValue()); //option 1
        //System.out.print(((char)Integer.parseInt(data.get(i).toString())); //option 2
    }
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

## 2.2 FileWriter

Per escriure a un fitxer de text farem servir la classe **FileWriter**. El mètode **write()** ens escriu contingut en un fitxer, tenim diferents mètodes que provenen de diferents classes i subclasses, podem escriure un caràcter (en format ASCII), una cadena de caràcters o un *String*:

| Mètode                                                                         | Prové de la classe | Significat                                                                    |
|--------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------|
| <b>void</b> writer( <b>int</b> c)                                              | OutputStreamWriter | Escriu un caràcter en valor ASCII                                             |
| <b>void</b> writer( <b>char</b> [] chars)                                      | Writer             | Escriu una cadena de caràcters                                                |
| <b>void</b> writer( <b>char</b> [] chars, <b>int</b> begin, <b>int</b> length) | OutputStreamWriter | Escriu una porció de la cadena de caràcters, a partir de begin i longitud end |
| <b>void</b> writer(String s)                                                   | Writer             | Escriu un <i>String</i>                                                       |
| <b>void</b> writer(String s, <b>int</b> begin, <b>int</b> length)              | OutputStreamWriter | Escriu una part del <i>String</i>                                             |

Nota: També tenim el mètode **append()** que afegeix un caràcter al final del fitxer.

Passos per escriure en un fitxer:

- Instanciem un objecte de la classe **FileWriter** que serà el nostre fitxer de text. Indiquem **true** si volem escriure al final del fitxer, **false** si volem sobreescriure el contingut del fitxer.
- Recorrem un a un els caràcters que volem escriure.
- Amb el mètode **write()** escrivim cada un dels caràcter en el fitxer. Degut a què aquest mètode llança una excepció del tipus **IOException**, hem de capturar-la amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Tanquem l'escriptura amb el mètode **close()**.

**Exemple 1:** Escriure un *String* al final del fitxer iterant el *String*

```
String st = "\nWrite this sentence into file";
try {
    FileWriter file = new FileWriter("text.txt", true); //creates a file if it doesn't exists
    for (int i = 0; i < st.length(); i++) {
        file.write(st.charAt(i));
    }
    file.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

**Exemple 2:** Escriure un *String* al final del fitxer directament sense iterar

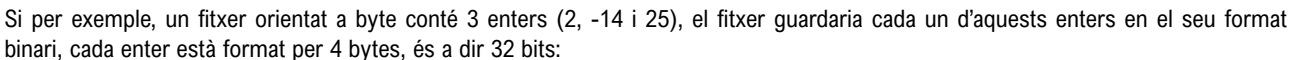
```
String st = "\nWrite this sentence into file";
try {
    FileWriter file = new FileWriter("text.txt", true); //creates a file if it doesn't exists
    file.write(st);
    file.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

**Exemple 3:** Escriure una part del *String* (de la posició 0 fins la 9) al final del fitxer directament sense iterar.

```
String st = "\nWrite this sentence into file";
try {
    FileWriter file = new FileWriter("text.txt", true); //creates a file if it doesn't exists
    file.write(st, 0, 10);
    file.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

**Exemple 4:** Escriu el contingut d'un **ArrayList** al final del fitxer, *dataC* és la llista de caràcters que hem creat a l'apartat 1.

```
List<Character> dataC = new ArrayList<>();
Collections.addAll(dataC, 'A', 'Z', 'C', 'T', 'O', '\n', 'V');
try {
    FileWriter file = new FileWriter("text.txt", true); //creates a file if it doesn't exists
    for (int i = 0; i < dataC.size(); i++) {
        file.write(dataC.get(i));
    }
    file.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```



25: 0000000000000000000000000000000011001

## 4.1 FileInputStream

Per llegir un fitxer de bytes farem servir la classe **InputStream** i la subclasse **FileInputStream**.

Els mètodes són els mateixos que la classe **FileReader**.

| Mètode                                                     | Exemple                                                                               | Prové de la classe | Significat                                                                                                                                                                                      |
|------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int read()</code>                                    | <code>int c = fr.read()</code>                                                        | InputStreamReader  | Llegeix un caràcter en codi ASCII.                                                                                                                                                              |
| <code>int read(char[] chars)</code>                        | <code>char[] chars = new char[10];<br/>int totalChar = fr.read(chars);</code>         | Reader             | Guarda a <code>chars</code> (array de 10 caràcters) els 10 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 10).                        |
| <code>int read(char[] chars, int start, int length)</code> | <code>char[] chars = new char[40];<br/>int totalChar = fr.read(chars, 10, 30);</code> | InputStreamReader  | Guarda a <code>chars</code> (array de caràcters) a partir de la posició 10 els 30 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 30). |

Exemple de com podem llegir un fitxer:

- Hem de crear una col·lecció d'enters per anar guardant les dades.
- Instanciem un objecte de la classe **FileInputStream** que serà el nostre fitxer.
- Degut a què aquest mètode llança una excepció **FileNotFoundException**, hem de capturar-la amb un **try-catch**, aquesta excepció controla si el fitxer existeix.
- Recorrem el fitxer fins trobar el final de fitxer (-1).
- Amb el mètode **read()** llegim un byte i l'afegim a la col·lecció.
- Degut a què aquest el mètode **read()** llança una excepció del tipus **IOException**, hem de capturar-la amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Tanquem la lectura amb el mètode **close()**.

Codi exemple:

```
List <Integer> data=new ArrayList<>();
try {
    FileInputStream fileInput = new FileInputStream("drawing.jpg");
    boolean endOfFile = false;
    while (!endOfFile) {
        int byteData = fileInput.read();

        data.add(byteData);
        if (byteData == -1) {
            endOfFile=true;
        }
        System.out.println(byteData);
    }
    fileInput.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //in case file has been moved, deleted, ...
    System.out.println("Error: " + e.getMessage());
}
```

## 4.2 UTF-8

Per poder llegir un fitxer de tipus text amb caràcters amb format Unicode **UTF-8** (accents, caràcter especials, etc) com per exemple els caràcters en català o castellà, o caràcters matemàtics, podem fer servir la classe **InputStreamReader** o **FileInputStream** en comptes de la classe **FileReader**:

### 4.2.1 Utilitzant InputStreamReader

Aquesta classe té un constructor que podem indicar-li el tipus de codificació, que s'especifica amb la classe **Charset**. Primer definim el tipus de caràcter amb la classe **Charset**, utilitzant el mètode **forName()**, i després instanciem un **InputStreamReader** indicant-li l'objecte que hem creat del tipus **Charset**:

```
try {
    Charset inputCharset = Charset.forName("UTF-8");

    FileInputStream fileInput = new FileInputStream("text.txt");
    InputStreamReader isr = new InputStreamReader(fileInput, inputCharset);

    boolean endOfFile = false;
    while (!endOfFile) {
        int data = isr.read();

        System.out.print((char) data);
        if (data == -1) {
            endOfFile=true;
        }
    }
    fileInput.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //in case file has been moved, deleted, ...
    System.out.println("Error: " + e.getMessage());
}
```

### 4.2.2 Utilitzant FileInputStream

- Instanciem un objecte de la classe **FileInputStream** que serà el nostre fitxer.
- Definim un **buffer** de mida 1024 (per exemple) que és un array de tipus **byte**.
- Amb el mètode **read(buffer)** guardarem a **buffer** el contingut del fitxer (només 1024 chars)
- Convertim el que s'ha llegit en un *String* indicant el format Unicode **UTF-8**.

Codi exemple:

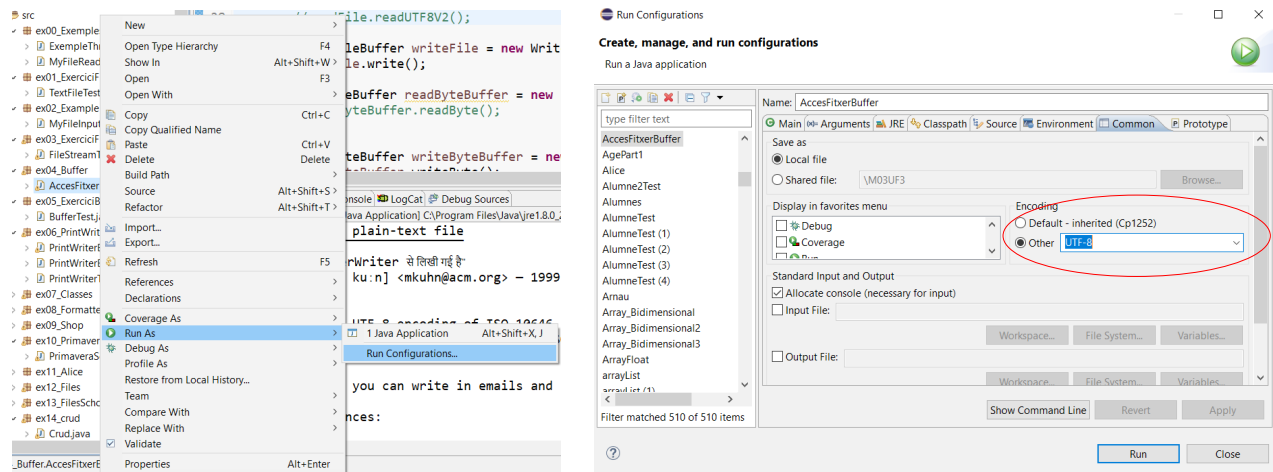
```
try {
    FileInputStream fileInput = new FileInputStream("text.txt");

    boolean endOfFile = false;
    while (!endOfFile) {
        byte[] buf = new byte[1024];
        int byteData = fileInput.read(buf);

        String value = new String(buf, StandardCharsets.UTF_8);
        System.out.println(value);
        if (byteData == -1) {
            endOfFile=true;
        }
    }
    fileInput.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Una altra manera d'executar fitxers de tipus text des d'Eclipse amb caràcters en format Unicode UTF-8, és seleccionant el tipus de codificació alhora d'executar el programa (*Encoding*):

Amb el botó dret a sobre del programa, seleccionem *Run As > Run Configuration*  
I a la pestanya *Common*, seleccionar *Other = UTF-8*



### 4.3 FileOutputStream

Per escriure a un fitxer de bytes farem servir la classe **OutputStream** i la subclasse **FileOutputStream**.

| Mètode                                                           | Prové de la classe | Significat                                                                            |
|------------------------------------------------------------------|--------------------|---------------------------------------------------------------------------------------|
| <b>void</b> <code>writer(byte[] b)</code>                        | FileOutputStream   | Escriu un array de tipus byte                                                         |
| <b>void</b> <code>writer( byte b)</code>                         | FileOutputStream   | Escriu un byte                                                                        |
| <b>void</b> <code>writer(byte[] b, int begin, int length)</code> | FileOutputStream   | Escriu una porció de l'array de tipus byte, des de la posició begin i llargada length |

Exemple de com podem escriure en un fitxer un String, un enter, un caràcter i un double:

- Instanciem un objecte de la classe **FileOutputStream** que serà el nostre fitxer de sortida. Cal capturar l'excepció **FileNotFoundException** que llança aquesta classe.
- Convertim les nostres dades a un array de tipus byte.
- Amb el mètode **write()** escrivim l'array de tipus byte al fitxer. Degut a què aquest mètode llança una excepció **IOException**, hem de capturar-la amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Tanquem l'escriptura amb el mètode **close()**.

Codi exemple:

```
try {
    FileOutputStream fileOutput = new FileOutputStream(file);

    //Write String
    String text = "THIS IS A NEW LINE\n";
    byte[] byteString = text.getBytes();
    fileOutput.write(byteString);

    //Write int
    int num = 125;
    byte[] byteInt = Integer.toString(num).getBytes();
    fileOutput.write(byteInt);

    //Write char
    char c = 'z';
    byte[] byteChar = Character.toString(c).getBytes();
    fileOutput.write(byteChar);

    //Write double
    double doub = 1250.55;
    byte[] byteDouble = Double.toString(doub).getBytes();
    fileOutput.write(byteDouble);

    fileOutput.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

## 5. Throws

Les excepcions les hem recollit amb un **try-catch**, però també es poden declarar en el **main** o a la declaració del mètode amb la paraula reservada **throws**.

| Amb try-catch                                                                                                                                                                                                                                                                                                                                                          | Amb throws                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>try {     FileReader file = new FileReader("file.txt");     int c=file.read();     while (c!=-1) {         char cl = (char) c;         System.out.print(cl);         c=file.read();     }     file.close(); } catch (FileNotFoundException e) {     System.out.println(e.getMessage()); } catch (IOException e) {     System.out.println(e.getMessage()); }</pre> | <pre>public static void main(String[] args) throws FileNotFoundException,     IOException {     FileReader file = new FileReader("file.txt");     int c=file.read();     while (c!=-1) {         char cl = (char) c;         System.out.print(cl);         c=file.read();     }     file.close(); }</pre> |

### Note: FileWriter Character Encoding

The **FileWriter** assumes that you want to encode the bytes to the file using the default character encoding for the computer your application is running on. This may not always be what you want, and you cannot change it!

If you want to specify a different character encoding scheme, don't use a **FileWriter**. Use an **OutputStreamWriter** on a **FileOutputStream** instead. The **OutputStreamWriter** lets you specify the character encoding scheme to use when writing bytes to the underlying file.

## Appendix

Per obrir un fitxer orientat a byte en un editor hexadecimal, podeu utilitzar algun programa que permeti veure el contingut del fitxer en la seva representació en bytes. Hi ha diverses eines que poden fer això, aquí t'esmento algunes opcions:

HxD: És un editor hexadecimal gratuït per a Windows fàcil d'utilitzar. Pots descarregar-lo des del seu lloc web oficial: <https://mh-nexus.de/en/hxd/>.

Hex Fiend: És un editor hexadecimal gratuït per a Mac que té una interfície d'usuari simple. Pots descarregar-lo des de la Mac App Store o des del lloc web oficial: <https://hexfiend.com/>.

Bless: És un editor hexadecimal gratuït i de codi obert per a Linux. Pots descarregar-lo des del lloc web oficial: <https://github.com/bwrsandman/Bless>.

Quan hagi descarregat i instal·lat el programa de la teva elecció, simplement obre'l i carrega l'arxiu que desitges editar en format hexadecimal. Veureu una representació de cada byte al fitxer en forma de nombres hexadecimal i caràcters ASCII o Unicode.

## Llicència

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is in <http://www.gnu.org/licenses/fdl.txt>